sol: model interface

sol

Models

A Sol version is a Python module `sol.config.sol_n`. It defines the model, its input requirements, and its training objective. Let B denote the batch size.

Module Fields

Field	Value	Semantics
`model`	`nn.Module`	Model instance; each forward pass returns named output tensors.
`inputs`	`list[str]`	Nonempty, ordered input tensor names; determines positional argument order.
`dtype`	`torch.dtype`	The version's floating-point dtype.
`training_params`	`dict`	Training settings and execution device; overrides trainer defaults.
`calculate_loss`	callable	Receives the output dictionary and batch; returns the training loss.
`datasets`	`list[dict]`	Training only: tensor dictionaries in train, validation, test order.

Inputs and Batches

Datasets and batches are `dict[str, Tensor]`. All fields share a leading example axis; position i identifies the same example in every field. Only names in `inputs` are forwarded to the model. Other fields, including targets, remain available to the loss function.

```
predictions = sol.model(*(batch[key] for key in sol.inputs))
loss = sol.calculate_loss(predictions, batch)
```

Input features keep the same meaning in training and inference. Shared preparation constructs the requested tensors; new features can be added conditionally on `inputs`. Inputs and model parameters use the configured execution device.

Named Outputs

Every `model.forward` and `inference.predict` call returns `dict[str, Tensor]` containing `"win_logits"`. Each output preserves the input example order and its leading batch axis, including when $B = 1$.

Key	Shape	Semantics
`"win_logits"`	$(B,)$	Floating-point Blue-win logits. For logit z_i, $P_i(\text{Blue}) = \sigma(z_i)$.
Other names	$(B, \dots)$	Additional predictions; each head defines its dtype, trailing shape, and meaning.

Inference preserves the output dictionary and logits. Consumers select heads by name; adding a head preserves the meanings of existing keys.

```
win_logits = inference.predict(sol, drafts)["win_logits"]
```

Loss

`calculate_loss(predictions, batch)` receives all heads and returns a floating tensor of shape (), differentiable during training. It is a mean over the batch's examples: the trainer aggregates $\sum_j B_j L_j / \sum_j B_j$ across batches. The version defines its target fields, objectives, and head weights. Benchmarks define their own metrics on the selected heads.